

HTTP Status Codes — Complete Real-Time DevOps Interview Explanation

HTTP status codes are used by servers to tell:

- Request success or failure
- Authentication issues
- Server problems
- Timeout issues

In real-time DevOps work:

- We analyze these codes in logs
 - Monitor APIs
 - Troubleshoot applications
 - Debug ingress/load balancer issues
-

HTTP Status Code Categories

Category	Meaning
1xx	Informational
2xx	Success
3xx	Redirection
4xx	Client-side Errors
5xx	Server-side Errors

1xx — Informational Responses

What Does It Mean?

Request received,
server still processing.

Example

100 Continue

Server says:

Continue sending request body

Real-Time Usage

Rarely seen in normal troubleshooting,
mostly internal HTTP communication.

Important Interview Point

“1xx responses indicate intermediate communication states.”

2xx — Success Responses

What Does It Mean?

Request processed successfully.

200 OK

Meaning

Application responded successfully.

Real-Time Scenario

curl http://app-service

Response:

200 OK

Means:

- Application healthy
 - API reachable
-

Important DevOps Usage

Used in:

- Health checks
 - Monitoring
 - Readiness probes
-

201 Created

Meaning

New resource created successfully.

Real-Time Scenario

POST API creates:

- User
- Record
- Deployment

Returns:

201 Created

Important Interview Point

“2xx codes indicate successful communication between client and application.”

3xx — Redirection Responses

What Does It Mean?

Client redirected to another location.

Common Example

301 Moved Permanently

Used during:

- Domain migration
 - HTTPS redirection
-

Real-Time Scenario

HTTP traffic redirected to HTTPS.

Ingress returns:

Important DevOps Usage

Common in:

- Nginx
 - Load balancers
 - Ingress controllers
-

4xx — Client-Side Errors

Meaning

Problem from:

Client/request side

400 Bad Request

Meaning

Invalid request sent.

Causes

- Wrong JSON payload
- Invalid parameters
- Missing request body

Real-Time Scenario

Frontend sending malformed JSON.

Backend returns:

400 Bad Request

How DevOps Troubleshoots?

Check:

- API logs
 - Request payload
 - Reverse proxy logs
-

Important Interview Point

“400 errors usually indicate malformed client requests rather than infrastructure issues.”

401 Unauthorized

Meaning

Authentication required or invalid token.

Causes

- Expired JWT token
 - Wrong credentials
 - Missing auth header
-

Real-Time Scenario

API requests failing because:

Expired access token

DevOps Troubleshooting

Check:

- OAuth/JWT validation
 - API Gateway auth
 - IAM permissions
-

403 Forbidden

Meaning

User authenticated,
but not authorized.

Difference Between 401 & 403

401	403
Authentication failed	Permission denied
User identity invalid	User lacks access

Real-Time Scenario

Developer authenticated successfully,
but lacks admin access.

API returns:

403 Forbidden

Important Interview Point

“403 errors are authorization failures, not authentication failures.”

404 Not Found

Meaning

Requested resource/path not found.

Causes

- Wrong URL
 - Wrong API route
 - Missing ingress path
 - Service unavailable
-

Real-Time Kubernetes Scenario

Ingress configured:

/api

User accessing:

/app

Returns:

404 Not Found

DevOps Troubleshooting

Check:

- Ingress rules
 - API routes
 - Nginx config
 - Service paths
-

Important Interview Point

“404 errors often occur due to routing or path configuration mismatches.”

5xx — Server-Side Errors

Meaning

Problem from:

Application/server side

500 Internal Server Error

Meaning

Application failed internally.

Causes

- Application crash
 - Database issue
 - Null pointer exception
 - Runtime error
-

Real-Time Scenario

Spring Boot app failed due to:

Database connection failure

Returns:

500 Internal Server Error

DevOps Troubleshooting

Check:

- Application logs
 - DB connectivity
 - Pod logs
 - JVM errors
-

Important Interview Point

“500 errors indicate backend application failure rather than client issues.”

502 Bad Gateway

Meaning

Gateway/proxy received invalid response from backend.

Common Components

- Nginx
 - Ingress Controller
 - Load Balancer
-

Causes

- Backend pod crash
 - Wrong service port
 - Application not listening
-

Real-Time Kubernetes Scenario

Nginx ingress forwarding traffic:

- Backend pod crashed

Ingress returns:

502 Bad Gateway

DevOps Troubleshooting

Check:

- Pod health
- Service endpoints
- Backend ports
- Readiness probes

Important Interview Point

“502 errors usually occur between proxy layer and backend application.”

503 Service Unavailable

Meaning

Application/service temporarily unavailable.

Causes

- No healthy pods
 - Application maintenance
 - Scaling delays
-

Real-Time Kubernetes Scenario

Deployment rollout happening:

- All pods restarting

Service temporarily unavailable:

503 Service Unavailable

DevOps Troubleshooting

Check:

- Pod readiness
 - Replica availability
 - HPA scaling
 - Service endpoints
-

Important Interview Point

“503 errors often indicate temporary service unavailability rather than permanent failure.”

504 Gateway Timeout

Meaning

Gateway timed out waiting for backend response.

Causes

- Slow backend API
- High CPU usage
- Database latency
- Long-running requests

Real-Time Scenario

API taking:

90 seconds

Ingress timeout configured:

60 seconds

Result:

504 Gateway Timeout

DevOps Troubleshooting

Check:

- Backend latency
 - Database performance
 - Pod CPU/memory
 - API timeout settings
-

Important Interview Point

“504 errors usually indicate backend slowness rather than application crash.”

Real-Time DevOps Troubleshooting Flow

User Reports API Issue



Check HTTP Status Code



4xx → Client/Request Issue
5xx → Server/Application Issue
↓
Check Logs
↓
Check Pods/Services
↓
Check DB/Network
↓
Fix Root Cause

Kubernetes + HTTP Error Mapping

Error	Common Kubernetes Cause
404	Wrong ingress path
502	Backend pod crash
503	No healthy pods
504	Slow backend response
500	Application exception

Real-Time Monitoring Usage

DevOps teams monitor:

- 5xx spike alerts
- API latency
- Error rate percentage
- Load balancer metrics

using:

- Prometheus
 - Grafana
 - ELK
 - Datadog
-

Most Important Interview Closing Line

“HTTP status codes help quickly identify whether issue originates from client requests, application logic, backend services, or infrastructure layers like ingress and load balancers.”